



ELTE EÖTVÖS LORÁND
UNIVERSITY

Algorithms and Data Structures I.

Gábor Pusztai

Department of Algorithms And Their Applications
Faculty of Informatics
ELTE - Eötvös Loránd University, Budapest

2026. február 15.

Content in this Practice I

1 Simple Selection algorithm

- Simple Selection - Max and Min Selection Sort
- Max Selection Sort - example
- Max Selection Sort - Analysis
- Min Selection Sort - Exercise

2 Simple Exchange algorithm

- Simple Exchange - Bubble Sort
- Bubble Sort - example
- Bubble Sort - Analysis
- Improved Bubble Sort - example
- Stable sorting algorithms

Simple Selection algorithm

Simple Selection - Introduction

- Illustration problem:
Sorting my favorite musics in an unsorted playlist.



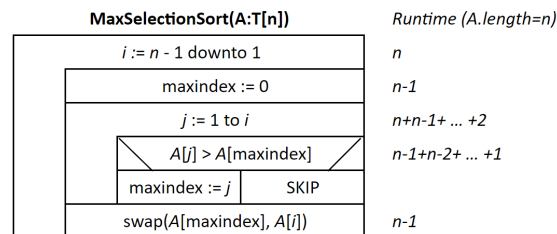
- Idea:
 - Find (*select*) the best song.
 - Put it to the top (first position) of the playlist.
 - Repeat! (second favorite to the second position and so on)

Simple Selection - Max and Min Selection Sort

- 2 types: **Max (or min) Selection Sort**
 - Select the maximum (or minimum) item in the unsorted part.
 - Swap it with the last (or first) positioned item in the unsorted part. (This data will be sorted)
 - Repeat these operations with the remaining unsorted items.
- *Selection sort is a simple and efficient sorting algorithm that works by repeatedly selecting the smallest (or largest) element from the unsorted portion of the array and moving it to the sorted portion of the array.*

Max Selection Sort - example

Array:	$\langle 5, 4, 2, 8, 3 \rangle$
1. iteration:	$\langle 5, 4, 2, \underline{8}, \underline{3} \rangle$
2. iteration:	$\langle \underline{5}, 4, 2, \underline{3} 8 \rangle$
3. iteration:	$\langle 3, \underline{4}, 2 5, 8 \rangle$
4. iteration:	$\langle \underline{3}, \underline{2} 4, 5, 8 \rangle$
5. iteration:	$\langle 2 3, 4, 5, 8 \rangle$
Sorted array:	$\langle 2, 3, 4, 5, 8 \rangle$



Max Selection Sort - Analysis

Time complexity analysis:

- Worst-case: when we need to sort an array in ascending order, but the array is initially in descending order. $O(n^2)$
- Best-case: when the array is already sorted. $\Omega(n^2)$

Space complexity analysis:

- No extra space is required $O(1)$

Min Selection Sort - Exercise

Min selection sort: is the mirror image of the Max selection sort.

Sort the following array with *Min selection sort*!

$\langle 44, 55, 12, 42, 94, 18, 06, 67 \rangle$

Example:

1. iteration: $\langle \underline{44}, 55, 12, 42, 94, 18, \mathbf{06}, 67 \rangle$

2. iteration: $\langle 06 | \underline{55}, \mathbf{12}, 42, 94, 18, 44, 67 \rangle$

3. iteration: $\langle 06, 12 | \underline{55}, 42, 94, \mathbf{18}, 44, 67 \rangle$

.....

Animation: https://upload.wikimedia.org/wikipedia/commons/3/3e/Sorting_selection_sort_anim.gif

Min Selection Sort - Exercise

Min selection sort: is the mirror image of the Max selection sort.

Sort the following array with *Min selection sort*!

$\langle 44, 55, 12, 42, 94, 18, 06, 67 \rangle$

Solution:

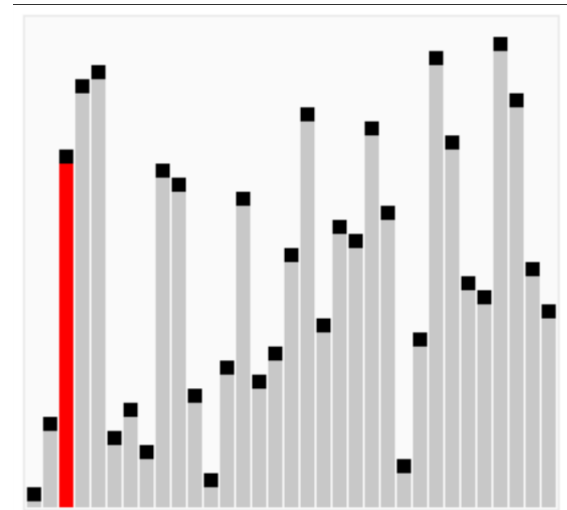
Initial keys	<u>44</u>	55	12	42	94	18	06	67
i=1	06	<u>55</u>	12	42	94	18	44	67
i=2	06	12	<u>55</u>	42	94	18	44	67
i=3	06	12	18	<u>42</u>	94	55	44	67
i=4	06	12	18	42	<u>94</u>	55	44	67
i=5	06	12	18	42	44	<u>55</u>	94	67
i=6	06	12	18	42	44	55	<u>94</u>	67
i=7	06	12	18	42	44	55	67	94

Simple Exchange algorithm

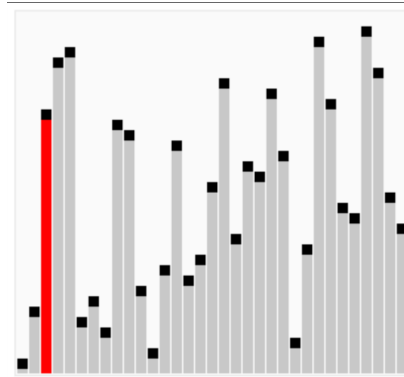
Simple Exchange - Introduction

- Illustration problem:

I have different items on my desk that I want to sort by height, but I don't know the exact height of each object.



Simple Exchange - Introduction



- Idea:
 - Place all the objects next to each other and compare the objects' height to each other
 - Swap if the left is smaller than the right neighbor.
 - Repeat! (Keep swapping until every item is in order)

Simple Exchange - Bubble Sort

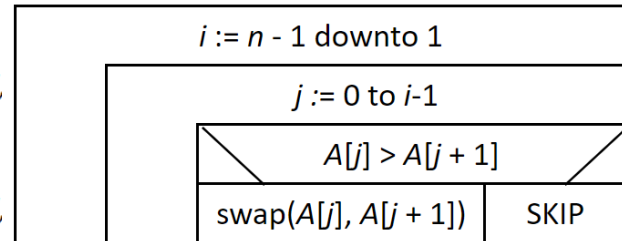
- Bubble Sort algorithm:
 - Traverse from the left and compare the adjacent 2 elements.
 - Swap if the next element is smaller. (The higher one is placed at the right side.)
 - In this way, the largest element is moved to the rightmost end at first. (This data will be sorted)
 - Repeat these operations with the remaining unsorted items.
- *Bubble sort is a simple algorithm that works by repeatedly swapping the adjacent elements if they are in the wrong order.*

Bubble Sort - example

0	1	2	3	4	swap
3	5	2	4	1	0
3	5	2	4	1	1
3	2	5	4	1	1
3	2	4	5	1	1
3	2	4	1	5	1. iteration end, 5 is sorted
3	2	4	1	5	1
2	3	4	1	5	0
2	3	4	1	5	1
2	3	1	4	5	2. iteration end,
2	3	1	4	5	0
2	3	1	4	5	1
2	1	3	4	5	3. iteration end,
2	1	3	4	5	1
1	2	3	4	5	4. iteration end

Overall swap: 7
Overall comparisons: 10

BubbleSort(A :T[n])



Bubble Sort - Analysis

Time complexity analysis:

- Worst-case: when we need to sort an array in ascending order, but the array is initially in descending order. $O(n^2)$
- Best-case: when the array is already sorted. $\Omega(n^2)$
But we can improve this! $\rightarrow$ *Improved Bubble Sort algorithm*

Space complexity analysis:

- No extra space is required $O(1)$

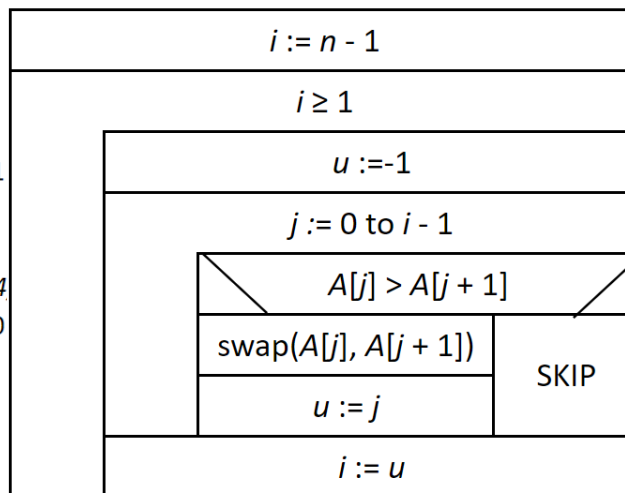
Animation: https://upload.wikimedia.org/wikipedia/commons/5/54/Sorting_bubblesort_anim.gif

Improved Bubble Sort - example

0	1	2	3	4	swap	
2	3	1	4	5	0	
2	3	1	4	5	1	u=1
2	1	3	4	5	0	
2	1	3	4	5	0	
2	1	3	4	5		1. iteration end 3,4
2	1	3	4	5	1	u=0
1	2	3	4	5		terminate

Overall swap: 2
Overall comparisons: 5

ImprovedBubbleSort($A : T[n]$)



Stable sorting algorithms

- A sorting algorithm is stable if given multiple records with the same values it keeps their original ordering.
- $\langle 1, 3, 1', 2, 6 \rangle \rightarrow \langle 1, 1', 2, 3, 6 \rangle$
- Insertion sort $\rightarrow$ **Stable**
- Bubble sort $\rightarrow$ **Stable**
- Maximum selection sort $\rightarrow$ **Not Stable**
Simple case example: $\langle 1, 2, 6, 5, 6' \rangle \rightarrow$ *watch out for the comparison operator for each direction of the inner loop*
 $\rightarrow$ Can be stable
Difficult case example: $\langle 1, 7, 6, 5, 6' \rangle \rightarrow$ *instead of swap, push the data* $\rightarrow$ Can be stable

Thank you for your attention!